Medians and Order Statistics & Elementary Data Structures

Student Name: Sindhuja Martha

Student ID: 005035378

Algorithms and Data Structures (MSCS-532-M80): **Assignment 6**

MS in Computer Science, University of the Cumberlands, KY

Professor: **Dr. Solomon**

March 14, 2026

Implementation and Analysis of Selection Algorithms and Elementary Data Structures

Abstract

This report details the implementation and analysis of selection algorithms and fundamental data structures. The initial section examines deterministic and randomized algorithms for identifying the k-th smallest element in an array. The deterministic approach employs the Median of Medians technique to ensure linear worst-case performance, whereas the randomized method utilizes the Quick select strategy with random pivot selection to achieve expected linear performance. Empirical results from program outputs validate the correctness of both implementations. The subsequent section investigates core data structures, including arrays, stacks, queues, and linked lists, evaluating their operational characteristics and analyzing the time complexity of associated operations. The findings indicate that careful algorithm design and data structure selection are critical determinants of computational efficiency and overall system performance.

Introduction

Order statistics algorithms help computer scientists quickly find items based on their position in a dataset. For example, they are used to find the median or the k-th smallest element. Fast algorithms are especially useful in data analysis, databases, and machine learning.

There are two main ways to solve the selection problem: deterministic and randomized algorithms. Deterministic algorithms always finish in a set amount of time, even in the worst case. Randomized algorithms are usually faster on average. Each method has its own strengths, depending on the situation. Basic data structures like arrays, stacks, queues, and linked lists are

key for organizing and processing data efficiently. Knowing how they work and how well they perform is important for building reliable software. This report looks at how selection algorithms and basic data structures are used, compares their performance, shows program results, and gives examples of how they work in real systems (Cormen et al., 2022).

Part 1: Implementation and Analysis of Selection Algorithms

Implementation of Deterministic Selection

The deterministic selection algorithm implemented in this project follows the Median of Medians technique. The list is split into groups of five, and each group is sorted to find the middle value. These middle values form a new list, and the middle value of this new list is found the same way. This final middle value is used to split the original list. After picking the middle value, the list is split into three groups: numbers less than the middle value, numbers equal to it, and numbers greater than it. Depending on the value of k, the method then continues with the group that has the number you are looking for. This way, many numbers are removed in each step, which makes sure the process is always fast, even in the worst case (Cormen et al., 2022).

Implementation of Randomized Selection

This project uses a random selection method based on Quick select. Rather than choosing the same pivot each time, it randomly selects one item from the list as the pivot. The list is then divided into items smaller than, equal to, or larger than the pivot.

After dividing the list, the method checks which group contains the k-th smallest item and continues working with that group. Choosing the pivot randomly makes it less likely to end up with very uneven groups, so the method usually works faster (Cormen et al., 2022).

Performance Analysis

Time Complexity Analysis of Deterministic and Randomized Selection Algorithms

The deterministic selection method used in this project uses the Median of Medians approach to find the kth smallest item in a list. The method splits the list into groups of five, finds the middle value in each group, and then finds the middle value of those medians. This value is then used to split the list. The benefit of choosing the pivot this way is that it makes sure the list is split fairly evenly. The median of medians makes sure that many items are either bigger or smaller than the pivot. Because of this, a big part of the list can be left out in the next steps of the method.

The running time of the deterministic selection algorithm can be described using the recurrence relation:

$$T(n) = T(n/5) + T(7n/10) + O(n)$$

This recurrence represents the different steps involved in the algorithm. The term T(n/5) represents the time required to find the median of the medians, where the algorithm first divides the array into groups of five elements and then recursively computes the median of those group medians. The term T(7n/10) represents the largest possible remaining portion of the array after the partitioning step. Because the pivot is carefully chosen using the median-of-medians strategy,

at least 30% of the elements are guaranteed to be eliminated in each iteration, meaning the largest recursive subproblem contains at most 7n/10 elements. The O(n) term represents the linear work required during each step, which includes dividing the array into groups of five, computing the median of each group, and partitioning the array around the selected pivot. Solving this recurrence results in a linear time complexity of O(n). Because the pivot always ensures that a constant fraction of elements is discarded during each recursive step, the deterministic algorithm maintains linear performance even in the worst case.

Randomized Quick select picks a random pivot and divides the list into three groups: less than, equal to, and greater than the pivot. It then continues searching only in the relevant group. While a bad pivot could slow things down to O(n^2), this is rare due to random selection (Cormen et al., 2022). On average, Randomized Quick select runs in linear time, O(n), because partitions shrink quickly. The deterministic version always guarantees linear performance, while the randomized version is faster on average but can be slower in rare worst cases.

Space Complexity and Additional Overheads

Both algorithms use extra memory to split data and repeat steps. The deterministic algorithm does extra work to find a good pivot, splitting the list into groups of five and finding medians. This slows it down in practice, though its speed is still O(n). Using recursion and extra lists can make memory use as high as O(n), but usually only O(log n) space is needed. The randomized version is simpler, skipping the Median of Medians step, so it's often faster in practice (Cormen et al., 2022). Randomized Quickselect's memory use depends on implementation: recursion and extra arrays can use O(n), but in-place methods keep it lower.

Deterministic selection offers stronger guarantees but is more complex. Randomized selection is simpler and typically more efficient, though worst-case performance is not guaranteed.

Empirical Results

The deterministic selection algorithm and the randomized Quick select algorithm are tested with different input sizes to see how they perform in practice. For each algorithm, we measured how long it took to process arrays with 1,000, 5,000, and 10,000 elements. The purpose was to see how their running times change as the input size grows.

```
Running Selection Algorithm Performance Tests
Size: 1000
Deterministic Select Time: 0.000776
Randomized Quickselect Time: 0.000465
----------------------------------------
Size: 5000
Deterministic Select Time: 0.002858
Randomized Quickselect Time: 0.001367
----------------------------------------
Size: 10000
Deterministic Select Time: 0.008089
Randomized Quickselect Time: 0.002989
----------------------------------------
```

Figure above shows the Screenshot of VSCode Output - implementation of Selection algorithms

Input Size	**Deterministic Selection Time (seconds)**	**Randomized Quick select Time (seconds)**
1,000	0.000776	0.000465
5,000	0.002858	0.001367
10,000	0.008089	0.002989

Table shows the output values from VSCode Output

Empirical Analysis of Results

The results show both algorithms take longer as data increases, but time grows steadily. For all input sizes tested, the randomized Quick select algorithm was consistently faster than the deterministic version. This is because the deterministic algorithm spends extra time finding a balanced pivot, while the randomized one simply picks a random pivot, reducing overhead. Although the randomized method can be slower in rare worst cases, this almost never happened in our tests. Overall, randomized Quickselect is typically faster in practice, while the deterministic method offers stronger worst-case guarantees.

In summary, the experimental results match what theory predicts. Both algorithms scale roughly linearly with input size, but the randomized Quick select algorithm is usually faster in practice because it has less overhead and a simpler way of picking the pivot.

Part 2: Elementary Data Structures Implementation and Performance Analysis

Time Complexity Analysis of Basic Operations in Data Structures

This project explores the efficiency of four fundamental data structures: arrays, stacks, queues, and linked lists. The performance of these structures depends on how they handle core operations such as inserting, deleting, and accessing data. Arrays keep items in contiguous memory, allowing instant access by index (O(1)), but inserting or deleting elements, especially in the middle, can be slow (O(n)) since other elements must be shifted. Stacks follow the Last-In-First-Out principle; adding (push) and removing (pop) items from the top are both fast and always take constant time, O(1) (Goodrich et al., 2014).

Queues operate on a First-In-First-Out basis, with enqueue and dequeue operations also taking O(1) time if implemented efficiently, making them suitable for tasks where order matters. Linked lists store elements in nodes connected by pointers, which means inserting new items is flexible and efficient (O(1)) if the position is known, but searching or deleting requires traversing the list (O(n)). Understanding these strengths and trade-offs allows developers to choose the most suitable data structure for their specific needs and optimize performance in different scenarios.

Trade-Offs Between Arrays and Linked Lists for Stacks and Queues

Both arrays and linked lists can be used to implement stacks and queues, but they offer different advantages.

Arrays provide fast access to elements and are easy to implement. When used for stacks, arrays work very efficiently because push and pop operations occur at the end of the array. However, arrays have a fixed size unless dynamic resizing is implemented. If the stack grows beyond the array's capacity, resizing the array can be expensive (Goodrich et al., 2014). Linked lists provide dynamic memory allocation, which means they can grow or shrink without resizing. This makes them flexible for stacks and queues that may experience unpredictable growth. However, linked lists require additional memory to store pointer references, which increases memory usage compared to arrays.

In summary, arrays are generally preferred when memory size is predictable and fast access is required, while linked lists are preferred when dynamic resizing and frequent insertions or deletions are needed (Goodrich et al., 2014).

Comparison of Data Structure Efficiency in Different Scenarios

Some data structures work better than others depending on what kind of operations you need to perform. Arrays are best when you need to access elements by their index often, like in matrix operations or numerical calculations. Because you can get any element instantly, they are perfect for these jobs. Stacks are useful when you need to keep track of the most recent action, like the undo feature in text editors or managing function calls in a program. Queues are helpful in systems that schedule tasks, like printer job lines or network packet processing, where tasks need to be handled in the order they come in. Linked lists are good for situations where you often add or remove items, like managing changing sets of data (Goodrich et al., 2014).

Practical Applications of Data Structures

Basic data structures are important in many real-world computing systems. Arrays are used in scientific computing, image processing, and database indexing because they let you quickly find items. Stacks help with solving math expressions, checking code structure, and undoing steps in certain methods. They also keep track of function calls in programming languages using the call stack. Queues are used in operating systems and networking to handle processes and data packets in the order they arrive. Some examples are CPU scheduling, printer task management, and message queues in distributed systems. Linked lists are used in managing memory, showing connections in networks, and real-time systems when memory needs to be flexible. Because you can add or remove pieces without changing the whole structure, linked lists are good for handling data that changes often (Goodrich et al., 2014).

Choosing the Appropriate Data Structure

Choosing the right way to organize data depends on things like how much memory it uses, how fast it works, and how hard it is to set up. Arrays let you find items quickly, but they

need a chunk of memory and can be slow to add new items. Linked lists use memory more flexibly, but they need extra space to keep track of where each item is and are slower to access. Stacks and queues make some problems easier by limiting how you can get to the items inside. If you need something fast and fixed in size, arrays work well. If you need to add or remove items often, linked lists are more flexible (Goodrich et al., 2014).

Empirical Analysis of Results:

```
Stack Example
Pop: 20

Queue Example
Dequeue: 1

Linked List Example
5 -> 10 -> 15 -> None
```

Output from VSCode Terminal for Implementation of Data structures

The empirical results obtained from the program output confirm that the implemented data structures operate according to their expected behaviors. For example, in the stack, the output “Pop: 20” means the last element added was the first one removed. This shows the stack’s Last-In-First-Out (LIFO) property. The pop operation removes the top item right away, so it works quickly and always takes the same amount of time.

Similarly, the queue example shows “Dequeue: 1”, which confirms the First-In-First-Out (FIFO) behavior of queues where the first inserted element is removed first. The linked list output “5 -> 10 -> 15 -> None” shows that nodes were added and connected one after another. The traversal prints each node until it gets to the end of the list, shown by None. This confirms the linked list works correctly, where adding new items can be quick but going through the list

means checking each node one by one, so it takes longer as the list gets bigger (Kleinberg & Tardos, 2006).

Conclusion

This project implemented and analyzed both deterministic and randomized selection algorithms as well as several fundamental data structures. The deterministic Median of Medians algorithm guarantees worst-case linear time complexity, while the randomized Quick select algorithm achieves expected linear performance through random pivot selection. The empirical results obtained from the program confirm the correctness of both implementations.

The project also demonstrated the behavior and performance characteristics of arrays, stacks, queues, and linked lists. Each structure provides unique advantages depending on the operational requirements of the application. Understanding these algorithms and data structures is essential for designing efficient and scalable software systems.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Python*. Wiley.

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.